

Security Audit

Steer Protocol (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	23
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Steer Protocol partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Steer Finance is a decentralized protocol that provides modular infrastructure for building composable, data-driven Web3 applications across multiple blockchains. It offers automated liquidity management, on-chain task automation, non-custodial staking, and a data marketplace to enhance DeFi operations. With over 200 strategic partners, 1,800+ deployed apps, and support for 30+ networks, Steer Finance enables developers to create and monetize decentralized applications, vaults, and strategies while integrating off-chain data seamlessly.

Project Name: Steer Protocol

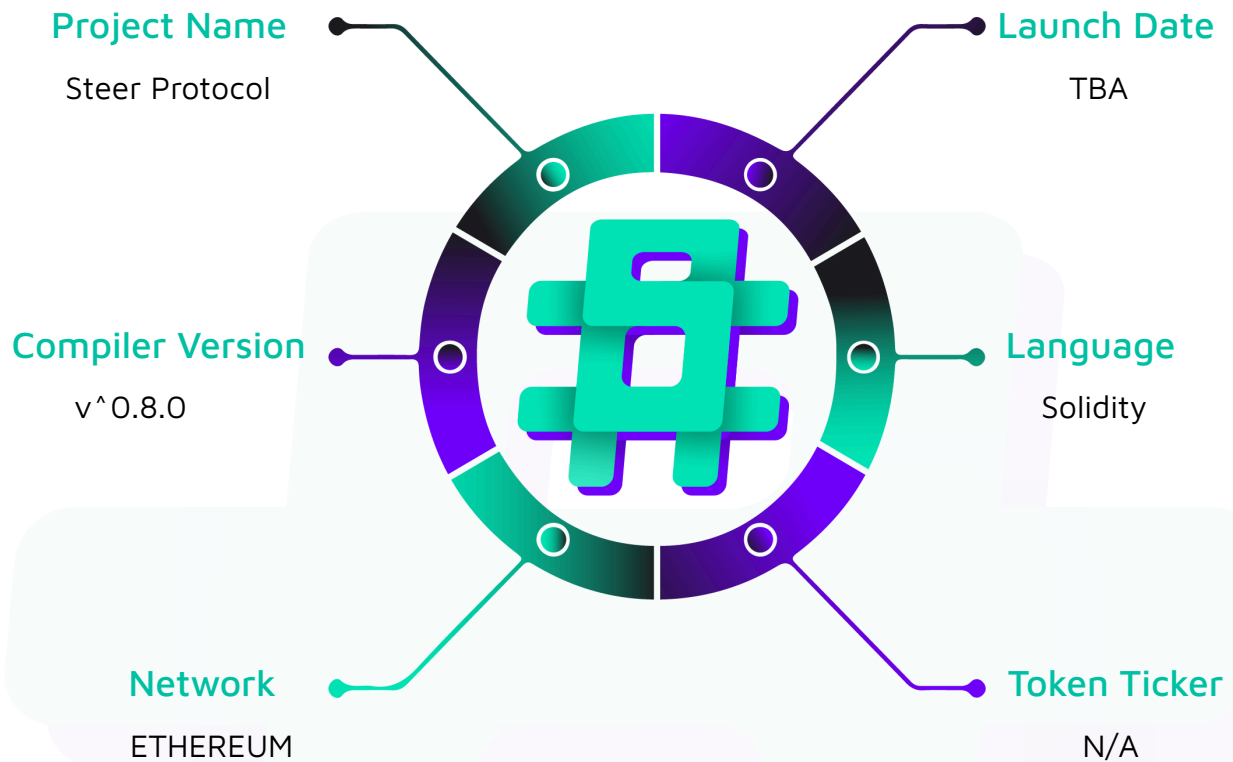
Project Type: DeFi

Compiler Version: ^0.8.0

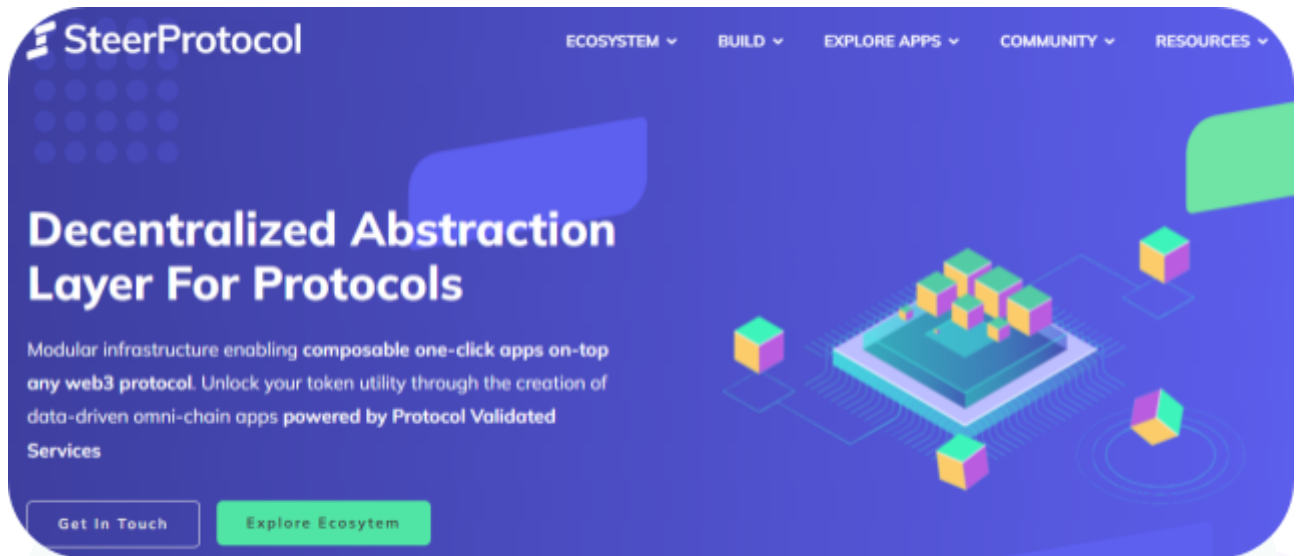
Website: <https://steer.finance/>

Logo:



Visualised Context:

Project Visuals:



Unlock Possibilities

Moonshot ideas, made possible

Fuse web3's innovation with web2's performance, empowering developers to build the unimaginable.

Our tools provide effortless access to any data source and lightning-fast secure compute.

Explore Apps



Audit Scope

We at Hashlock audited the solidity code within the Steer Protocol project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Steer Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	March, 2025
Contract 1	StrykeLPManager.sol
Contract 1 MD5 Hash	8d66a63e76b5e6a92b23488c95313029
Contract 2	StrykeLPManagerV2.sol
Contract 2 MD5 Hash	9c27fd4b7f751be43dc05e9d46963d5a
Contract 3	StrykeSteerLib.sol
Contract 3 MD5 Hash	5f7e9a2d076841d93f1eabc197ad3e4d
GitHub Commit Hash	17db3aefb541007f77d54c16bd50a4cf2ae313a

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

3 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
StrykeLPManager.sol Act as an automated liquidity manager for Stryke (formerly Dopex) liquidity positions Allow users to: - Deposit proportional amounts of two tokens and receive vault shares - Withdraw their proportional share of pool assets and accumulated fees Manage liquidity positions through: - Minting new positions on Stryke/Uniswap V3 - Burning existing positions when available - Reserving positions that are currently being used Handle fees through: - A total fee rate of 15% on earned fees - Split fees between Steer (66.67%) and strategist (33.33%) Provide safety features: - Pausable functionality - TWAP-based price deviation checks - Role-based access control for admin functions	Contract achieves this functionality.
StrykeLPManagerV2.sol Act as an automated liquidity manager for Stryke (formerly Dopex) liquidity positions Allow users to: - Deposit proportional amounts of two tokens and receive vault shares	Contract achieves this functionality.

- Withdraw their proportional share of pool assets and accumulated fees Manage liquidity positions through: - Minting new positions on Stryke/Uniswap V3 - Burning existing positions when available - Reserving positions that are currently being used Handle fees through: - A total fee rate of 15% on earned fees - Split fees between Steer (66.67%) and strategist (33.33%) Provide safety features: - Pausable functionality - TWAP-based price deviation checks - Role-based access control for admin functions	
StrykeSteerLib.sol - Allow automated management of Stryke liquidity positions - Handle position lending for options - Manage the lifecycle of positions from creation through usage to withdrawal - Ensure fair distribution of fees between all stakeholders - Provide safety mechanisms against potential exploits - Enable efficient liquidity management through off-chain computation and on-chain execution	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Steer Protocol project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Steer Protocol project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] StrykeLPManager, StrykeLPManagerV2 - Death Row Position Inflation Leading to Gas Exhaustion & Potential DoS

Description

The `StrykeLPManager` and `StrykeLPManagerV2` contracts can experience an accumulation of positions in its death row array over time, which could lead to increased gas costs and potential DoS scenarios. When users withdraw from positions that are currently being used by Stryke options, these positions are moved to a "death row" array during the next `tend()` operation. These positions remain in death row until they become unused AND their cooldown period expires.

Vulnerability Details

The accumulation occurs due to several factors:

- Positions are only moved to death row during `tend()` operations
- Positions only clear from death row when the position becomes unused (option expires/closes)
- The cooldown period has expired

Multiple unique tick ranges can accumulate since:

- Different users can withdraw from different positions
- Positions can have varying states of use and cooldown
- Normal Withdrawals themselves don't check or enforce cooldown periods

Proof Of Concept

```
function testDeathRowAccumulation() public {

    // Setup multiple users with different positions

    address[] memory users = new address[](5);

    uint256[] memory sharesByUser = new uint256[](users.length);

    // Initial setup same as before

    for(uint i = 0; i < users.length; i++) {

        users[i] = address(uint160(i + 100));

        vm.prank(rich_WETH);

        IERC20(WETH).transfer(users[i], 5e20);

        vm.prank(rich_USDC);

        IERC20(USDC).transfer(users[i], 5e10);

    }

    // Setup permissions first

    address posManOwner = 0x880C3cdCA73254D466f9c716248339dE88e4a97D;

    for(uint i = 0; i < users.length; i++) {

        vm.prank(posManOwner);

        positionManager.updateWhitelistHandlerWithApp(address(handler), users[i], true);

        vm.prank(posManOwner);

        handler.updateWhitelistedApps(users[i], true);

    }

    // Initial deposits and approvals

    for(uint i = 0; i < users.length; i++) {

        vm.startPrank(users[i]);

        IERC20(WETH).approve(address(vault), type(uint256).max);

        IERC20(USDC).approve(address(vault), type(uint256).max);

    }

}
```

```

IERC20(WETH).approve(positionManagerAddr, type(uint256).max);

IERC20(USDC).approve(positionManagerAddr, type(uint256).max);

IERC20(WETH).approve(address(handler), type(uint256).max);

IERC20(USDC).approve(address(handler), type(uint256).max);

(uint256 shares,,) = vault.deposit(2e18, 4000e6, 0, 0, users[i]);

sharesByUser[i] = shares;

vm.stopPrank();

}

// Record initial state

(int24[] memory initialLower,, uint256[] memory initialLiquidity) =
vault.getFullPositions(true);

// Multiple epochs following the working pattern
for(uint epoch = 0; epoch < 3; epoch++) {

    // Create positions through tend first

    tendWithNewRange(-197610 + int24(int256(epoch) * 150));

    // Now use positions for all users

    for(uint i = 0; i < users.length; i++) {

        int24 baseTick = -197610 + int24(int256(epoch) * 150);

        useDopexPosition(

            pool, hook, users[i], WETH, USDC, positionManagerAddr, handler,

            baseTick, baseTick + 10, uint128(1e15)

        );

    }

    swapSim();

    // Do withdrawals

    for(uint i = 0; i < users.length; i++) {

        vm.prank(users[i]);

```



```

        vault.withdraw(sharesByUser[i] / 3, 0, 0, users[i]);
    }

    // Tend to move positions to death row
    tendWithNewRange(-197610 + int24(int256(epoch + 1) * 150));

    // Unuse all positions
    for(uint i = 0; i < users.length; i++) {

        int24 baseTick = -197610 + int24(int256(epoch) * 150);

        unuseDopexPosition(

            pool, hook, users[i], positionManagerAddr, handler,

            baseTick, baseTick + 10, uint128(1e15)

        );
    }

    vm.warp(block.timestamp + 1500); // Stagger cooldown periods

    (int24[] memory currentLower,, uint256[] memory currentLiquidity) =
    vault.getFullPositions(true);

    emit log_named_uint("Death row positions after epoch", currentLower.length);
}

// Verify accumulation

(int24[] memory finalLower,, uint256[] memory finalLiquidity) =
    vault.getFullPositions(true);

    assertGt(finalLower.length, initialLower.length, "Death row should accumulate
    positions");

    assertGt(sumArray(finalLiquidity), sumArray(initialLiquidity), "Death row liquidity
    should increase");
}

function sumArray(uint256[] memory arr) internal pure returns (uint256 sum) {

    for(uint i = 0; i < arr.length; i++) {

        sum += arr[i];
    }
}

```

```

    }
}

function tendWithNewRange(int24 baseTick) internal {

    int24[] memory lowerTick = new int24[](1);

    int24[] memory upperTick = new int24[](1);

    uint16[] memory relativeWeight = new uint16[](1);

    uint256[] memory liquidity = new uint256[](1);

    lowerTick[0] = baseTick;

    upperTick[0] = baseTick + 10;

    relativeWeight[0] = 100;

    liquidity[0] = 1000000;

    IStrikeSteerLib.LiquidityPositions memory newPositions =
    IStrikeSteerLib.LiquidityPositions({

        lowerTick: lowerTick,

        upperTick: upperTick,

        relativeWeight: relativeWeight,

        liquidity: liquidity

    });

    vm.prank(orchestrator);

    vault.tend(10000, newPositions, abi.encode(0, 0));

}

```

Output:

Death row positions after epoch: 1

Death row liquidity: 5000000000000000

Death row positions after epoch: 2

Death row liquidity: 5000000000000005

Death row positions after epoch: 3

Impact

Each additional position in death row increases gas costs during `tend()` which could lead to high gas costs in popular vaults and may cause transaction failures if gas limits are reached

Recommendation

1. Implement a maximum limit on death row array size
2. Add logic to consolidate similar tick ranges in death row
3. Consider a batch clearance mechanism for expired death row positions
4. Add monitoring/alerts for death row size growth
5. Consider implementing a queue structure for death row positions to ensure FIFO processing

Status

Resolved

[M-02] StrykeLPManager#initialize, StrykeLPManagerV2#initialize - Missing zero address check for pool value

Description

The contracts retrieve the Uniswap pool address based on token0 and token1. However, it does not validate whether the retrieved pool address is non-zero before interacting with it. This introduces a critical vulnerability where the contract could attempt to interact with a non-existent pool, leading to unexpected behavior or transaction failures.

Vulnerability Details

```
pool = IUniswapV3Pool(  
    IUniswapV3Factory(0x1F98431c8aD98523631AE4a59f267346ea31F984).getPool(  
        params.token0, params.token1, params.poolFee
```

```
)  
);
```

If the pool between `params.token0` and `params.token1` doesn't exist, then the `getPool` will return zero and the `pool` variable is updated with zero address.

Impact

If the pool address is zero (indicating that no pool exists for the given token pair), any subsequent interaction with the pool will fail, causing the transaction to revert.

Recommendation

Before assigning with the pool, the contract should include a validation step to ensure that the retrieved pool address is non-zero.

Status

Resolved

QA

[Q-01] StrykeLPManager, StrykeLPManagerV2 - Lack of Input Validation in tend

Description

The `tend` function in `StrykeLPManager` lacks comprehensive input validation for its parameters, which could lead to unexpected states or inefficient operations

1. No validation that `newPositions` arrays (`lowerTick`, `upperTick`, `relativeWeight`) have matching lengths
2. No bounds checking on `totalWeight` - while there is a check in `_setBins` that uses this value, front-running validation would be more gas efficient
3. No validation of the relationship between the tick ranges in `newPositions`
4. No empty array validation for `newPositions` components

Recommendation

Add input validation at the start of the function

Status

Acknowledged

[Q-02] Contracts - Floating pragma

Description

The contracts have `pragma solidity ^0.8.0` and it might allow the contracts to be deployed with a different version than the one used for testing.

Different pragma versions being used in test and mainnet may pose unidentified security issues.

Recommendation

Specify a specific version of Solidity in the pragma statement.

Status

Resolved

[Q-03] Contracts - Inconsistent Use of uint and uint256 Types**Description**

The smart contracts use both uint and uint256 types interchangeably. In Solidity, uint is an alias for uint256, meaning they are functionally identical. However, using both types inconsistently within the same contract can lead to confusion for developers, reducing code readability and maintainability. This inconsistency may also increase the likelihood of errors during future updates or integrations.

Recommendation

Standardize the use of types throughout the contract. It is recommended to use uint256 explicitly, as it is more descriptive and aligns with best practices in Solidity development.

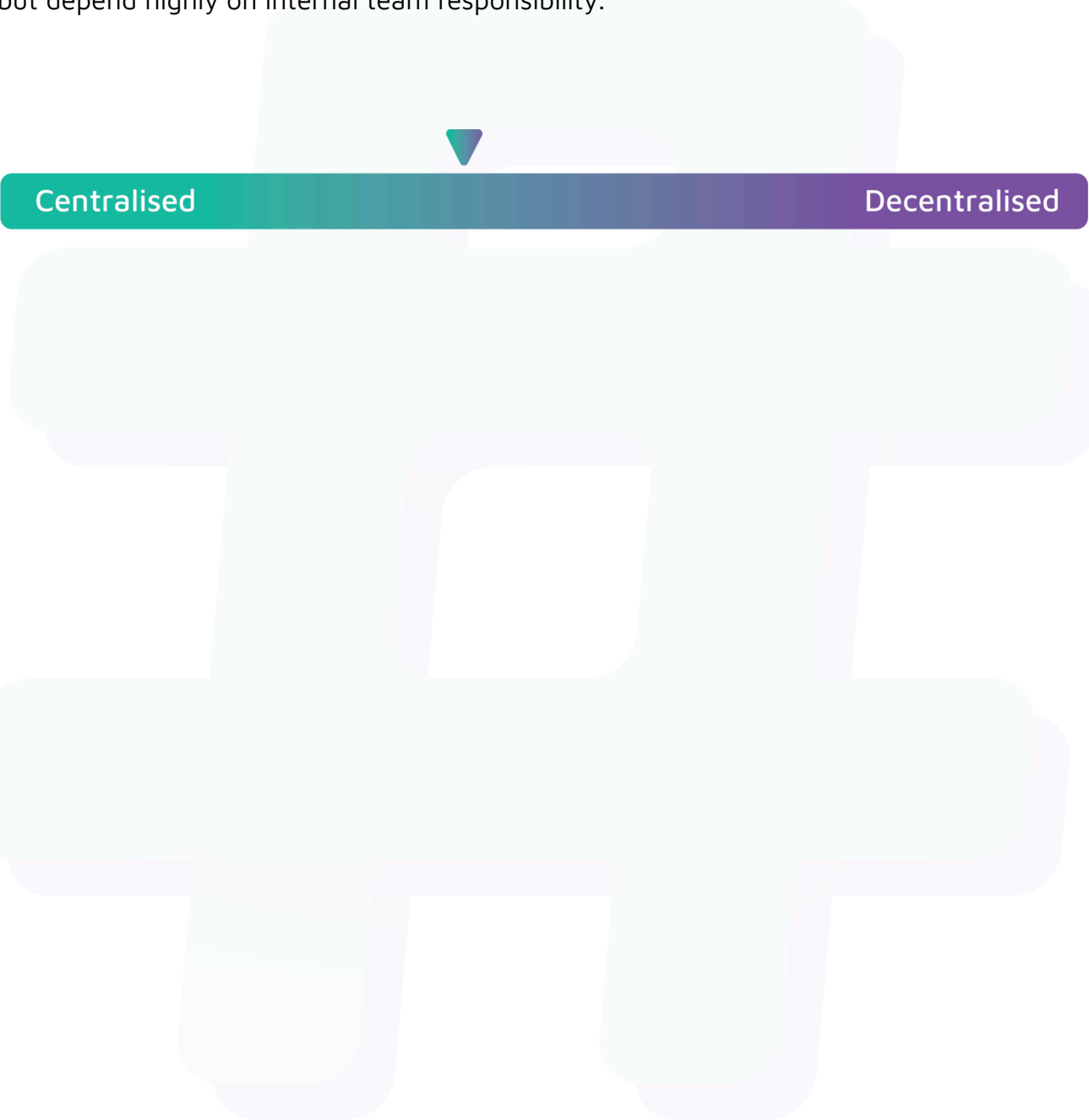
Status

Resolved

Centralisation

The Steer Protocol project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Steer Protocol project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.